

Review

- ❑ Last Class:

- ❑ Static parallelism
- ❑ Dynamic parallelism

- ❑ Today's class:

- ❑ Tomasulo

Scoreboard information (3 tables)



- **Instruction status:**
 - issued, read operands and started execution (dispatched), completed execution or wrote result,
- **Functional unit status** (assuming non-pipelined units)
 - busy/not busy
 - operation (if unit can perform more than one operation)
 - destination register - F_i
 - source registers (containing source operands) - F_j and F_k
 - the unit producing the source operands (if stall to avoid RAW hazards) - Q_j and Q_k
 - flags to indicate that source operands are ready -- R_j and R_k
- **Register result status:**
 - indicates the functional unit that contains an instruction which will write into each register (if any)

Limitations of the scoreboard approach

- No forwarding
- Structural hazards are cleared before instruction “issue”
- WAW and WAR hazards are cleared before instruction “issue”
- Did not discuss control hazards
- Execution (function) units are not pipelined

Possible enhancement

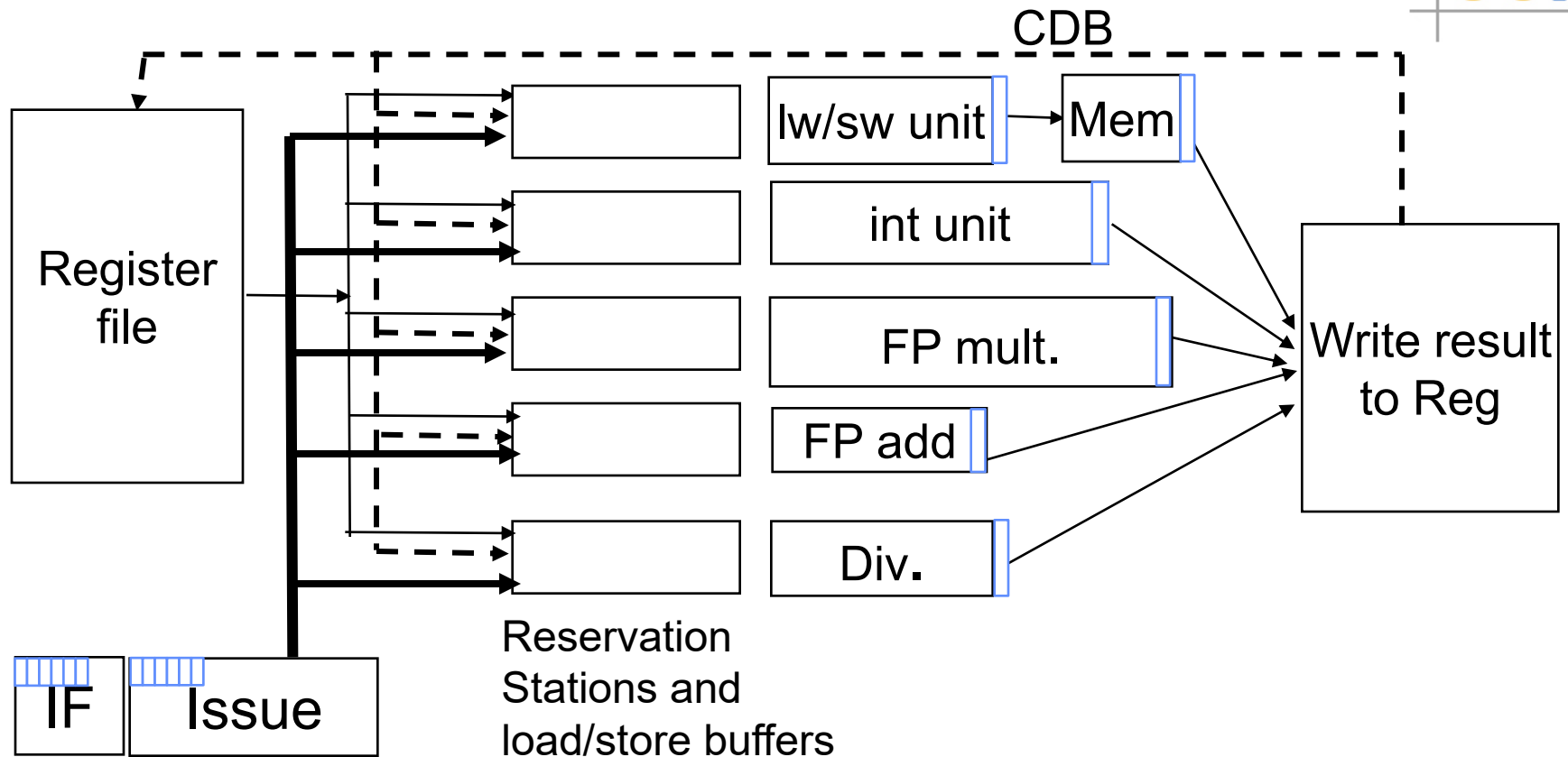
- If we can have “k” write-backs to registers per cycle and “k” parallel buses between registers and pipeline units, then
 - k functional units may be released per cycle
 - k instructions may be dispatched per cycles.
 - k instructions may be issued per cycle.

Need to extend the scoreboard to the case where the execution (function) units are pipelined

The Tomasulo approach

- Introduced for IBM 360/91 in 1966
- Main improvements over the scoreboard approach:
 - Uses forwarding on a Common Data Bus (CDB) -- more efficient dealing with RAW hazards,
 - avoids WAR hazards by reading the operands in the instruction-issue order, instead of stalling at issue. To accomplish this an instruction reads an available operand before waiting for the other.
 - Later version avoids WAR hazard by register renaming
 - Avoids WAW hazards by renaming the registers (using the *id* of a reservation station rather than the register *id*)
 - The control information and logic are distributed to the functional unit and not centralized.

The architecture for the Tomasulo scheme



- Each reservation station has an *id* and is used by one instruction during the lifetime of this instruction.
- Each unit has one or more reservation stations
- Reservation stations play the role of temporary registers (renaming)

Bookkeeping in Tomasulo's algorithm



- **Instruction status:**
 - issued, executing or writing result,

Instruction	Issue	Execute	Write result

Bookkeeping in Tomasulo's algorithm



- **Reservation stations (functional units) status:**
 - busy/not busy
 - operation (if unit can perform more than one operation)
 - source operands (data values) - V_j and V_k
 - the *reservation stations* producing the source operands (if stall to avoid RAW hazards) - Q_j and Q_k
 - Address field, A , for load/store buffers (store effective address)

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1							
Load2							
Add1							
Add2							
Add3							
Mult1							

Bookkeeping in Tomasulo's algorithm



- **Register result status:**

- indicates the *reservation station* that contains an instruction which will write into each register (if any)

	F0	F2	F4	F6	F8	F10	F12	...	F30
Qi									

Three stages of control



• Issue

- If a reservation station is available for the needed functional unit
 - » read ready operands (V_j/V_k)
 - » for operands that are not ready, rename the register to the reservation station that will produce it (Q_j/Q_k),
- Reservation stations for load/store instructions are called load/store buffers.

• Execution.

- Monitor the CDB for the operand that is not ready,
- When both operands are available, execute.
- If more than one station per unit, only one station can start execution.
- Do not start execution before previous branches have completed.

• Write result.

- Write to CDB (and to registers) -- may be a structural hazards if only one CDB bus.
- Make the reservation station (the functional unit) available.

Load and store instructions:

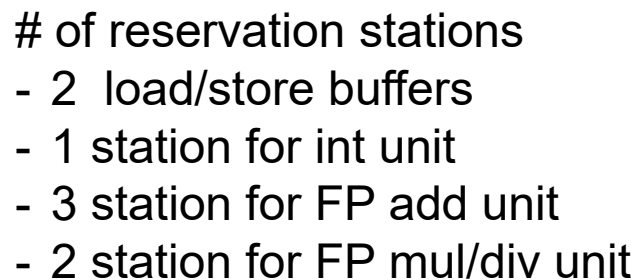
- Uses load/store buffers, and each buffer is like a reservation station.
- Address calculation (put result in buffer), then memory operation
- The result of a load is put on the CDB
- Stores are executed in program order (loads in any order)
- Performs memory disambiguation between store and load buffers
 - Hardware logic figures out whether a load is safe to execute given the stores already in the store buffer.

Load and store instructions:

- Uses load/store buffers, and each buffer is like a reservation station.
- Address calculation (put result in buffer), then memory operation
- The result of a load is put on the CDB
- Stores are executed in program order (loads in any order)
- Performs memory disambiguation between store and load buffers
 - Hardware logic figures out whether a load is safe to execute given the stores already in the store buffer.

Remarks:

- May have more reservation stations than registers
- The original Tomasulo algorithm was introduced before caches were incorporated into commercial processors
- If more than one issued instruction writes into a register, only the last one does the actual write (no WAW hazards).



Instruction		Issue	Execute	Write result
fld	F6, 34(R2)	X	X	
fld	F2, 45(R3)	X	X	
fmul.d	F0, F2, F4	X		
fsub.d	F8, F2, F6	X		
fdiv.d	F10, F0, F12	X		
fadd.d	F6, F8, F2	X		

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	Y	Load					34+Reg[R2]
Load2	Y	Load					45+Reg[R3]
Add1	Y	Sub			Load2	Load1	
Add2	Y	Add			Add1	Load2	
Add3	no						
Mult1	Y	Mul		Reg[F4]	Load2		
Mult2	Y	Div		Reg[F12]	Mult1		
Int	no						

	F0	F2	F4	F6	F8	F10	F12	...	F30
Qi	Mult1	load2		Add2	Add1	Mult2			

Instruction		Issue	Execute	Write result
fld	F6, 34(R2)	X	X	X
fld	F2, 45(R3)	X	X	
fmul.d	F0, F2, F4	X		
fsub.d	F8, F2, F6	X		
fdiv.d	F10, F0, F12	X		
fadd.d	F6, F8, F2	X		



Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	no						
Load2	Y	Load					45+Reg[R3]
Add1	Y	Sub			Load2		
Add2	Y	Add			Add1	Load2	
Add3	no						
Mult1	Y	Mul		Reg[F4]	Load2		
Mult2	Y	Div		Reg[F12]	Mult1		
Int	no						

	F0	F2	F4	F6	F8	F10	F12	...	F30
Qi	Mult1	load2		Add2	Add1	Mult2			

Instruction		Issue	Execute	Write result	
fld	F6, 34(R2)	X	X	X	
fld	F2, 45(R3)	X	X	X	done
fmul.d	F0, F2, F4	X			
fsub.d	F8, F2, F6	X			
fdiv.d	F10,F0,F12	X			
fadd.d	F6, F8, F2	X			

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	no						
Load2	no						
Add1	Y	Sub	Reg[F2]	Reg[F6]	Load2		
Add2	Y	Add		Reg[F2]	Add1	Load2	
Add3	no						
Mult1	Y	Mul	Reg[F2]	Reg[F4]	Load2		
Mult2	Y	Div		Reg[F12]	Mult1		
Int	no						

	F0	F2	F4	F6	F8	F10	F12	...	F30
Qi	Mult1	Load2		Add2	Add1	Mult2			

Instruction		Issue	Execute	Write result
fld	F6, 34(R2)	X	X	X
fld	F2, 45(R3)	X	X	X
fmul.d	F0, F2, F4	X	X	
fsub.d	F8, F2, F6	X	X	X
fdiv.d	F10,F0,F12	X		
fadd.d	F6, F8, F2	X		

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	no						
Load2	no						
Add1	no						
Add2	Y	Add	Reg[F8]	Reg[F2]	Add1		
Add3	no						
Mult1	Y	Mul	Reg[F2]	Reg[F4]			
Mult2	Y	Div		Reg[F12]	Mult1		
Int	no						

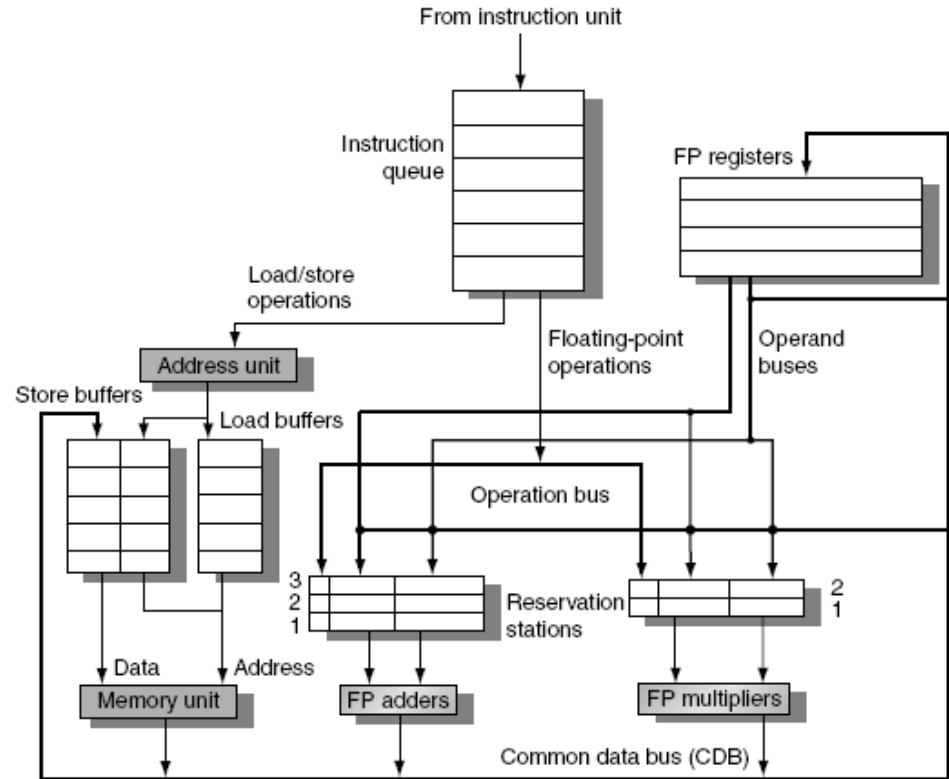
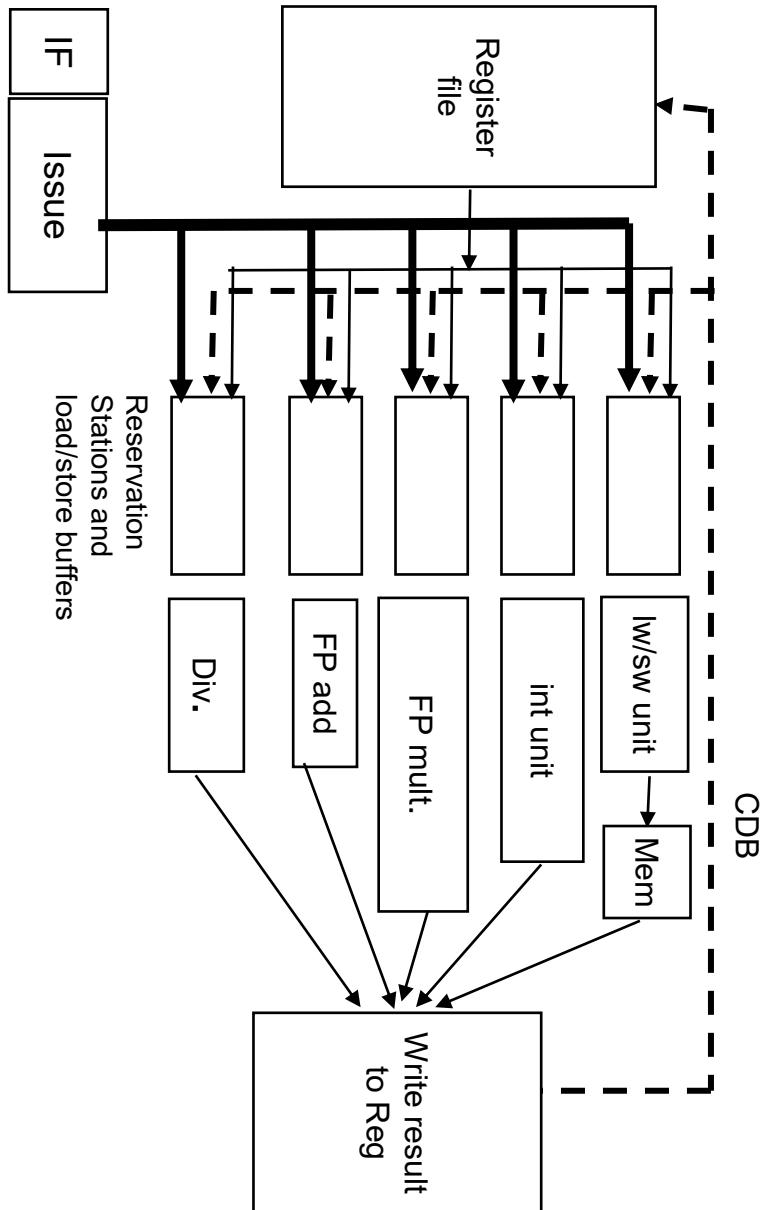
	F0	F2	F4	F6	F8	F10	F12	...	F30
Qi	Mult1			Add2	Add1	Mult2			

Instruction		Issue	Execute	Write result	
fld	F6, 34(R2)	X	X	X	done
fld	F2, 45(R3)	X	X	X	done
fmul.d	F0, F2, F4	X	X		
fsub.d	F8, F2, F6	X	X	X	done
fdiv.d	F10,F0,F12	X			
fadd.d	F6, F8, F2	X	X	X	

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	no						
Load2	no						
Add1	no						
Add2	no						
Add3	no						
Mult1	Y	Mul	Reg[F2]	Reg[F4]			
Mult2	Y	Div		Reg[F12]	Mult1		
Int	no						

	F0	F2	F4	F6	F8	F10	F12	...	F30
Qi	Mult1			Add2		Mult2			

Figure 3.10



Basic OOO Instruction Flow Overview

1. Fetch (**in program order**): **Fetch** multiple sequential instructions in parallel from the IM (I\$)
2. Decode, Rename, & Dispatch (**in program order**)
3. Issue (**Out Of Order -- OOO**): When an instr has all of its source data and the FU (Functional Unit) it needs is free, it is issued for **execution**
4. Writeback (**OOO**): When the dst value has been computed it is written back to the PhysicalRegFile and other shared structures – the instr **completes**
5. Commit (**in program order**): Changes the state of the machine

Enabling OoO Execution

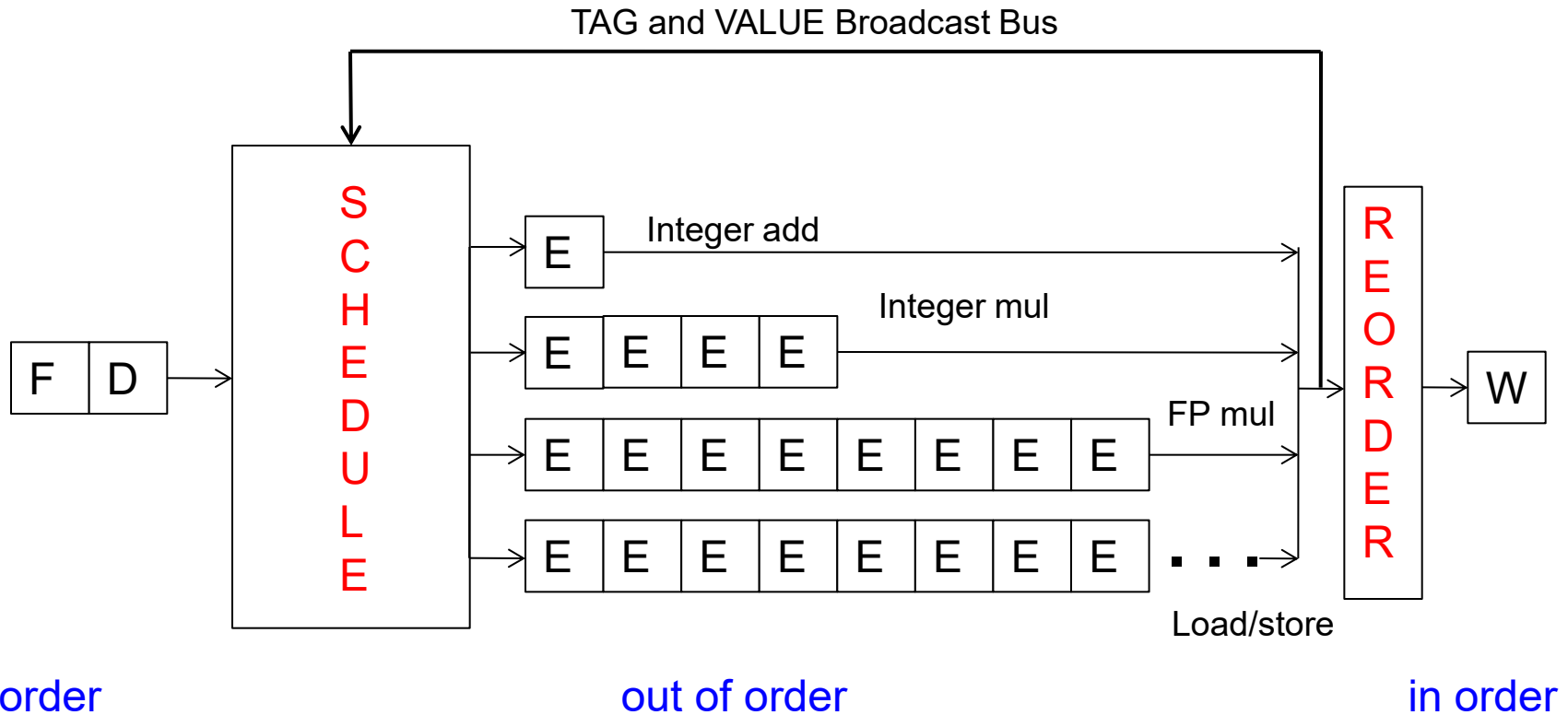
1. Need to link the consumer of a value to the producer
 - ❑ **Register renaming:** Associate a “tag” with each data value
2. Need to buffer instructions until they are ready
 - ❑ Insert instruction into **reservation stations** after renaming
3. Instructions need to keep track of readiness of source values
 - ❑ **Broadcast the “tag”** when the value is produced
 - ❑ Instructions **compare their “source tags”** to the broadcast tag → if match, source value becomes ready
4. When all source values of an instruction are ready, issue the instruction to functional unit (FU)
 - ❑ What if more instructions become ready than available FUs?

Hardware-based Speculation (§3.6)



- The goal is to allow instructions after a branch to start execution before the outcome of the branch is confirmed.
- There should be *no* consequences (including exceptions) if it is determined that the instruction should not execute.
 - Use dynamic branch prediction and use OOO execution
 - Use a Reorder Buffer (ROB) to reorder instructions after execution
 - Commit results to registers and memory in-order
 - All un-committed results can be flushed if a branch is miss-predicted
- Can free the reservation station when an instruction is in the reorder buffer.
- For each register, R, the register status table keeps the ROB number reserved by the instruction which will write into R (instead of the RES station number).

Two Humps in the Pipeline



- ❑ **Hump 1:** Reservation stations (scheduling window)
- ❑ **Hump 2:** Reordering (reorder buffer)

Reorder buffers (ROB)



- 3 fields: instruction, destination, value
- When issuing a new instruction, read a register value from the ROB if the status table indicates that the source instruction is in a ROB.
- Hence, ROB's supply operands between execution complete & commit => more virtual registers.
- ROB's form a circular queue ordered in the “issue order”.
- Once an instruction reaches the head of the ROB queue, it commits the results into register or memory.
- Hence, it's easy to undo speculated instructions on miss-predicted branches or on exceptions
- Should flush the pipe as soon as you discover a miss-predictions – all earlier instructions should commit

Steps of Speculative Tomasulo Algorithm

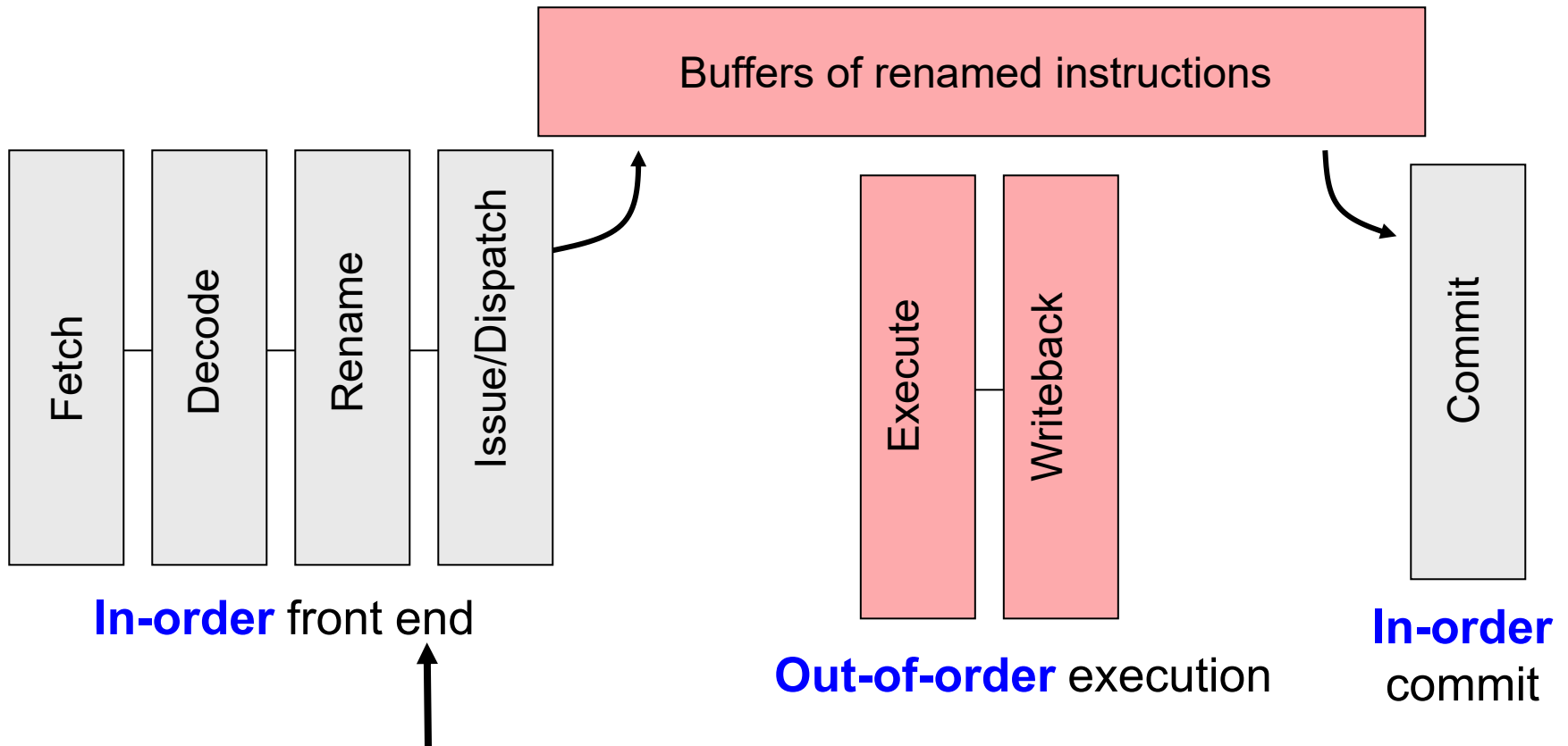


Combining branch prediction with dynamic scheduling

- **Issue** (sometimes called Dispatch)
 - If a RES station and a ROB are free, issue the instruction to the RES station after reading ready registers and renaming non-ready registers
- **Execution** (sometimes called issue)
 - When both operands are ready, then execute; if not ready, watch CDB for result; when both in reservation station, execute (checks RAW)
- **Write result** (WB)
 - Write on CDB to all awaiting RES stations & send the instruction to the ROB; mark reservation station available.
- **Commit** (sometimes called graduation)
 - When instruction is at head of ROB, update registers (or memory) with result and free ROB. A miss-predicted branch flushes all non-committed instructions.

Out-of-Order Pipeline Progress

- ❑ Have completed Fetch, Decode, Rename (in program order) and are ready to Dispatch



Instr's now have unique register names, so can now put into OOO execution structures

Reservation stations status

Name	Busy	Op	Vj	Vk	Qj	Qk	Dest.	A
Load1	no							
Load2	no							
Add1	Y	sub	Reg[F2]	Reg[F6]			ROB4	
Add2	Y	add	Reg[F8]	Reg[F2]			ROB6	
Add3	no							
Mult1	Y	Mul	Reg[F2]	Reg[F4]			ROB3	
Mult2	Y	Div		Reg[F6]	ROB3		ROB5	

Reservation stations status

Name	Busy	Op	Vj	Vk	Qj	Qk	Dest.
Load1	no						
Load2	no						
Add1	Y	sub	Reg[F2]	Reg[F6]			ROB4
Add2	Y	add	Reg[F8]	Reg[F2]			ROB6
Add3	no						
Mult1	Y	Mul	Reg[F2]	Reg[F4]			ROB3
Mult2	Y	Div		Reg[F6]	ROB3		ROB5

ROB
status

Name	Busy	Instruction		State	Dest.	value
ROB1	no	fld	F6, 34(R2)	Commit	F6	xxx
ROB2	no	fld	F2, 45(R3)	Commit	F2	xxx
ROB3	yes	fmul.d	F0, F2, F4	Execute	F0	xxx
ROB4	yes	fsub.d	F8, F2, F6	Wrote result	F8	xxx
ROB5	yes	fdiv.d	F10, F0, F6	Issued/executing	F10	
ROB6	yes	fadd.d	F6, F8, F2	Wrote result	F6	xxx

Reservation stations status

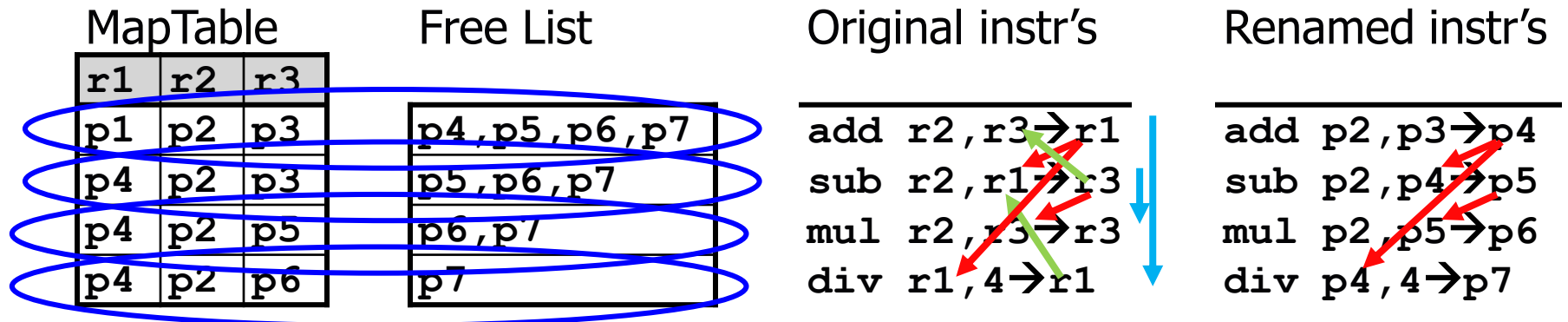
Name	Busy	Op	Vj	Vk	Qj	Qk	Dest.
Load1	no						
Load2	no						
Add1	Y	sub	Reg[F2]	Reg[F6]			ROB4
Add2	Y	add	Reg[F8]	Reg[F2]			ROB6
Add3	no						
Mult1	Y	Mul	Reg[F2]	Reg[F4]			ROB3
Mult2	Y	Div		Reg[F6]	ROB3		ROB5

ROB status	Name	Busy	Instruction		State		Dest.	value		
	ROB1	no	fld	F6, 34(R2)	Commit		F6	xxx		
	ROB2	no	fld	F2, 45(R3)	Commit		F2	xxx		
	ROB3	yes	fmul.d	F0, F2, F4	Execute		F0	xxx		
	ROB4	yes	fsub.d	F8, F2, F6	Wrote result		F8	xxx		
	ROB5	yes	fdiv.d	F10, F0, F6	Issued/executing		F10			
	ROB6	yes	fadd.d	F6, F8, F2	Wrote result		F6	xxx		
Register status	F0		F2	F4	F6	F8	F10	F12	...	F30
	Qi	ROB3		ROB6		ROB4	ROB5			

Note: reservation stations Add1 and Add2 may be released after the instructions finish execution.

Register renaming

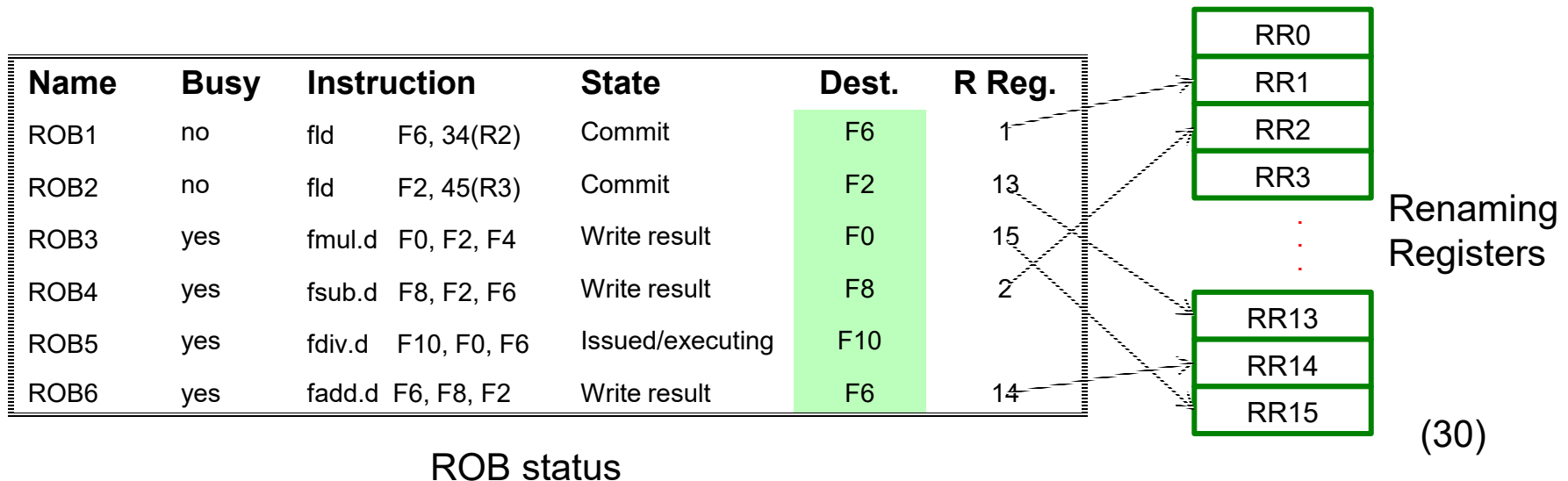
- ❑ To eliminate (WAW, WAR) register conflicts/hazards
- ❑ “Architected” vs “Physical” registers – level of indirection
 - ❑ Architected (ISA) register names: $r1, r2, r3$
 - ❑ Physical register locations: $p1, p2, p3, p4, p5, p6, p7$
 - ❑ Original mapping: $r1 \rightarrow p1, r2 \rightarrow p2, r3 \rightarrow p3$, $p4-p7$ are “available”



- ❑ Renaming – conceptually write each register once
 - + Removes **false** dependences (WAW and WAR)
 - + Leaves **true** dependences (RAW) intact!
- ❑ When to *reuse* a physical register? After overwriting instr commits.

Register Renaming

- Common variation of speculative design
- Reorder buffer keeps instruction information but not the result
- Extend register file with extra *renaming registers* to hold speculative results
- Rename register allocated at issue;
- Renaming registers are physical registers.
- Operands read either from register file (real or speculative) or via Common Data Bus
- Advantage: operands are always from single source (extended register file)



Remind: Multiple issue processors (§3.7)



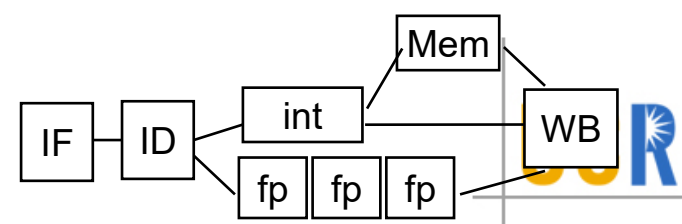
- Issue more than one instruction per clock cycle
 - Superscalar processors
 - » Statically scheduled (in-order execution)
 - » Dynamically scheduled (out-of-order execution)
- results in $CPI < 1$ (Instructions per clock, $IPC > 1$)
- The fetch unit gets an *issue packet* which contains multiple instructions
- The issue unit issues 1-8 instructions every cycle (usually, the issue unit is itself pipelined)
 - independent instructions
 - multiple resources should be available
 - branch prediction should be accurate
- Leads to multiple instruction completion per cycle

Dynamic scheduling (§3.8)

- Extends Tomasulo's algorithm to issue two (or more) instructions simultaneously to reservation stations.
- Either issue an instruction every half clock cycle, or double the logic to handle two instructions at once.
- Use the same logic. Some restrictions may be used to simplify hardware.
 - Example: issue only one FP and one int. operation every clock cycle. This reduces the load on the register files.
 - Do not issue dependent instructions in the same cycle
- To deal with control hazards without speculation (no ROBs): instructions following a branch can be issued but cannot start execution before the branch is resolved.
- Will look at the execution of

```
L1: fld      F0, 0(R1)
     fadd.d   F4, F0, F2
     fsd      F4, 0(R1)
     addi     R1, R1, -8
     bne      R1, R2, L1
```

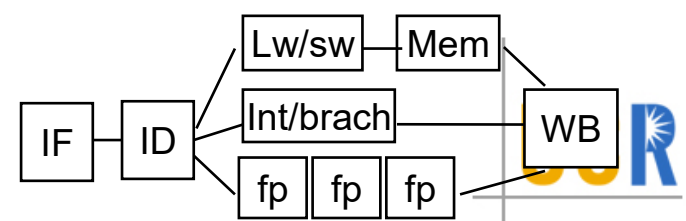

Dual issue with one int. and one FP add unit (not pipelined). Latency = 1, 2 and 3 for int. operations, loads and FP adds (one CDB).



Iteration	Instruction	Issued at	Executes	Mem access	Write CDB	comments
1	fld F0, 0(R1)	1	2	3	4	First issue
1	fadd.d F4, F0, F2	1	5		8	Wait for fld
1	fsd F4, 0(R1)	2	3	9		Wait for fadd.d
1	addi R1, R1, -8	2	4		5	Wait for ALU
1	bne R1, R2, L1	3	6			wait for addi
2	fld F0, 0(R1)	4	7	8	9	Wait for bne
2	fadd.d F4, F0, F2	4	10		13	Wait for fld
2	fsd F4, 0(R1)	5	8	14		Wait for fadd.d
2	addi R1, R1, -8	5	9		10	Wait for ALU
2	bne R1, R2, L1	6	11			wait for addi
3	fld F0, 0(R1)	7	12	13	14	Wait for bne
3	fadd.d F4, F0, F2	7	15		18	Wait for fld
3	fsd F4, 0(R1)	8	13	19		Wait for fadd.d
3	addi R1, R1, -8	8	14		15	Wait for ALU
3	bne R1, R2, L1	9	16			wait for addi

- No speculation: Instruction after *bne* cannot be issued in same cycle since the branch is not yet predicted (orange arrows)
- *fld*, *fsd*, *addi* and *bne* use the same integer Execute unit (see blue circles)
- Assume as many reservation stations as needed

If we have a separate int. unit for memory address calculation and we have a second CDB.

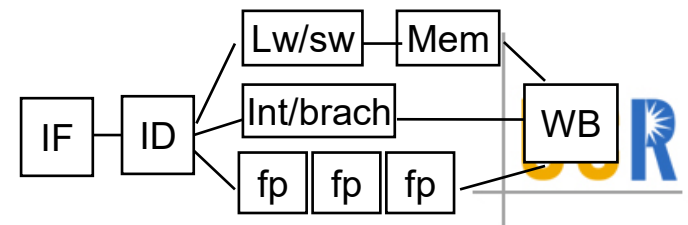


Iteration	Instruction	Issued at	Executes	Mem access	Write CDB	comments
1	fld F0, 0(R1)	1	2	3	4	First issue
1	fadd.d F4, F0, F2	1	5		8	Wait for fld
1	fsd F4, 0(R1)	2	3	9		Wait for fadd.d
1	addi R1, R1, -8	2	3		4	Exec. earlier
1	bne R1, R2, L1	3	5			wait for addi
2	fld F0, 0(R1)	4	6	7	8	Wait for bne
2	fadd.d F4, F0, F2	4	9		12	Wait for fld
2	fsd F4, 0(R1)	5	7	13		Wait for fadd.d
2	addi R1, R1, -8	5	6		7	Exec. earlier
2	bne R1, R2, L1	6	8			wait for addi
3	fld F0, 0(R1)	7	9	10	11	Wait for bne
3	fadd.d F4, F0, F2	7	12		15	Wait for fld
3	fsd F4, 0(R1)	8	10	16		Wait for fadd.d
3	addi R1, R1, -8	8	9		10	Exec. earlier
3	bne R1, R2, L1	9	11			wait for addi

No speculation: no instruction after *bne* can start before *bne* completes execution (cycle 5)

Multiple issue with speculation

Consider same example (as last slide)



Iteration	Instruction	Issued at	Executes	Mem access	Write CDB	comments
1	fld F0, 0(R1)	1	2	3	4	First issue
1	fadd.d F4, F0, F2	1	5		8	Wait for fld
1	fsd F4, 0(R1)	2	3	9		Wait for fadd.d
1	addi R1, R1, -8	2	3		4	
1	bne R1, R2, L1	3	5			wait for addi
2	fld F0, 0(R1)	4 3	6 4	7 5	8 6	No wait for bne
2	fadd.d F4, F0, F2	4	9 7		12 10	Wait for fld
2	fsd F4, 0(R1)	5 4	7 5	13 11		Wait for fadd.d
2	addi R1, R1, -8	5	6		7	
2	bne R1, R2, L1	6 5	8			wait for addi
3	fld F0, 0(R1)	7 6	9 7	10 8	11 9	No wait for bne
3	fadd.d F4, F0, F2	7 6	12 10		15 13	Wait for fld
3	fsd F4, 0(R1)	8 7	10 8	16 14		Wait for fadd.d
3	addi R1, R1, -8	8 7	9		10	
3	bne R1, R2, L1	9 8	11			wait for addi

Allow speculation, and may issue a branch with another instruction